What is an Algorithm?

An algorithm is a finite set of instructions that, if followed, accomplishes, a particular task.

In addition, all algorithms must satisfy the following criteria:

1. Input: Zero or more quantities are externally supplied.
2. Output: At least one quantity is produced
3. Definiteness: Each instruction is clear and unambiguous.
4. Finiteness: If we trace out the instructions of an algorithm, then for all cases, the algo. terminates after a finite number of steps.
5. Effectiveness: Every instruction must be very basic so that it can be carried out,

Program:

A program is the expression of an algorithm in a programming language. Such languages are designed so that each legitimate sentence has a unique meaning.

Sometimes words such as procedure, function, and subroutine are used synonymously for program

The study of algorithms includes many important and active areas of research. There are four distinct areas of study one can identify:

1. How to device algorithms.
2. How to validate algorithms.
3. How to analyze algorithms.
4. How to test a program

Shot on OnePlus
2025.08.06 11:28

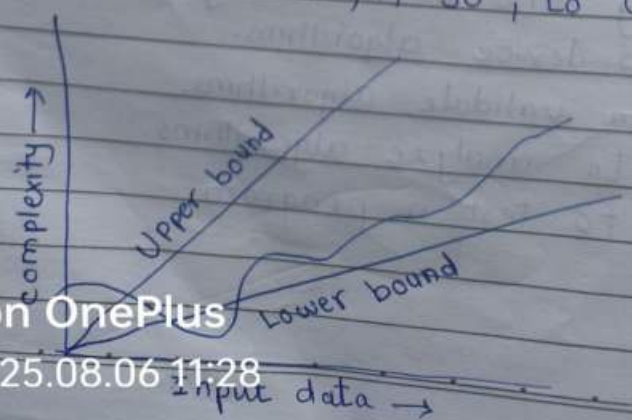
1. How to devise algorithms:
Creating an algorithm is an art which may never be fully automated
There are some important design techniques such as linear, non-linear, and integer programming are not covered here as they are traditionally covered

2. How to validate Algorithms:
Once an algo. is deviced, it is necessary to show that it computes the correct answer for all possible legal inputs. We refer to this process as algorithm validation.

3. How to Analyze algorithms -
This field of study is known as Analysis of Algorithms. As an algorithm is executed, it uses the computer's CPU to perform operations and its memory to hold the program and data.
AoA or performance analysis refers to the task of determining how much computing time and storage an algorithm requires.

4. How to test a program :-
Using debugging & profiling | performance measurement

debugging → process of executing programs on sample data sets to determine whether faulty result occur and, if so, to correct them.



Shot on OnePlus
2025.08.06 11:28

DS - How algorithm is performed in real world scenario with diff. data structure

AOA - systematic evaluation of how algo behaves concerning time complexity, space complexity & practical efficiency when manipulating diff. types of data.

What is more important than performance?

modularity	User-friendlyness
correctness	Programmer time
maintainability	Simplicity
functionality	Extensibility
Robustness	Reliability

Why study algorithms & performance?

- Algorithms help us to understand scalability.
- Performance often draws the line between what is feasible and what is impossible.
- Algorithmic mathematics provides a language for talking about program behavior.
- Performance is the currency of computing.

Asymptotic Notation

Worst Case	Best Case	Average Case
Big Oh Notation	Omega - Ω Notation	Theta Notation
O	Ω	Θ
Upperbound Function	Lower bound Function	Average

Shot on OnePlus
2025.08.06 11:28

Worst Case:
It is the longest time that an algorithm will user over all instances of size n for a given problem to produce a desired result.

Average Case:
It is the average time (or average space) that the algorithm will use over all instances of size n for a given problem to produce a desired result.

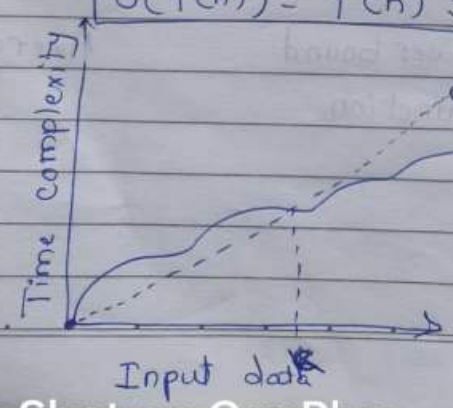
Best Case:
It is the shortest time (or least space) that the algorithm will use starting instances of size n for a given problem to produce a desired result.

Asymptotic Notations
It calculate the running time complexity of an algorith

1. Big Oh Notation, O - Worst Case
- The Notation O(n) is the formal way to express the upper bound of an algorithm's running time.
- It measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete.

$$\boxed{O(f(n)) = f(n) \leq c.g(n)}$$

f(n) → indicate given input
g(n) → worst case time complex of the algorithm



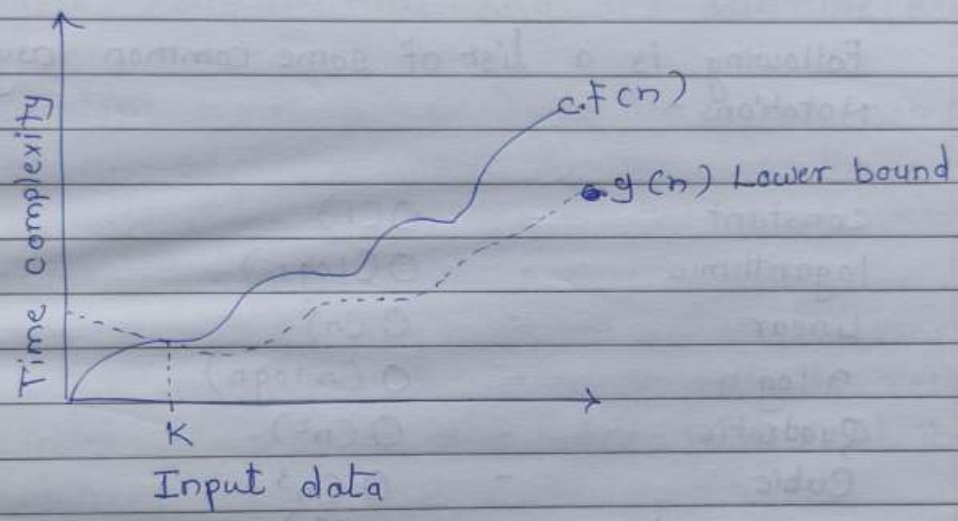
g(n) is an asymptotic upper bound for f(n)

Shot on OnePlus
✨ | 2025.08.06 11:28

2. Omega Notation, Ω → Best case

The notation $\Omega(n)$ is the formal way to express the lower bound of an algorithm's running time.
It measures the best case time complexity or the best amount of time an algorithm can possibly take to complete.

$$\boxed{\Omega(f(n)) = g(n) \leq c \cdot f(n)}$$



$f(n)$ → indicates given input
$g(n)$ → indicates best case time complexity of the algo.

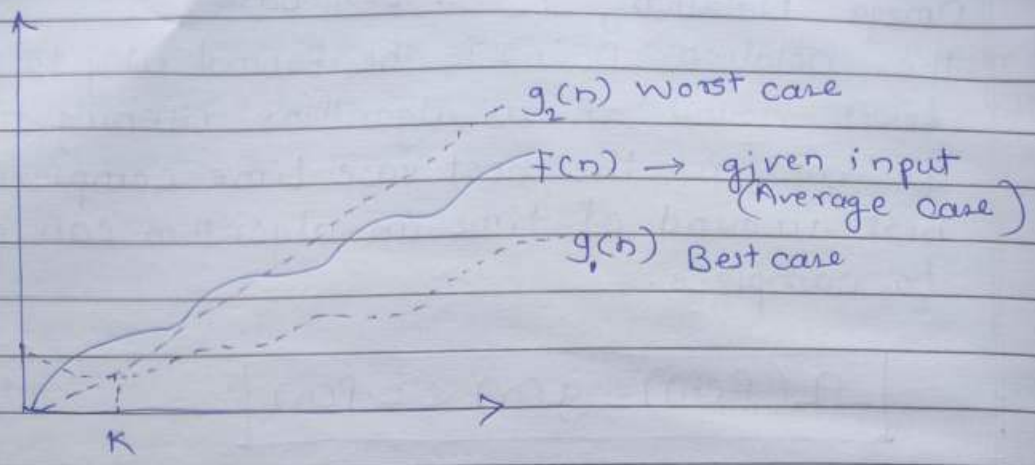
3. Theta Notation, Θ

The notation $\Theta(n)$ is a formal way to express both the lower bound & upper bound of an algorithm's running time with Average case time complexity

$$\Theta(f(n)) = O(f(n)) \;\&\; g(n) = \Omega(f(n))$$

Lower Bound $\leq$ Running Time $\leq$ Upper Bound

Shot on OnePlus
2025.08.06 11:28



Following is a list of some common asymptotic Notations -

Constant	-	$O(1)$
logarithmic	-	$O(\log n)$
Linear	-	$O(n)$
n log n	-	$O(n \log n)$
Quadratic	-	$O(n^2)$
Cubic	-	$O(n^3)$
polynomial	-	$n^{O(1)}$
Exponential	-	$2^{O(n)}$

Shot on OnePlus
✨ | 2025.08.06 11:28

Time Complexity

It deals with the quantification of the amount of time taken by a set of code or algorithm to process or run as a function of the amount of input.
Time complexity is defined as the number of times a particular instruction set is executed rather than the total time taken. It depends on some external Factors like the compiler used, the processor's speed etc.
The time T(P) taken by a program P is sum of the compiler time and the run time (execution)

1. Constant time, or O(1), is the time complexity of an algorithm that always uses the same no. of operations, regardless of no. of elements being operated on.
For ex-, an algorithm to return the length of a list may use a single operation to return the index number of the index number of the final element in the list.
O(1) doesnot imply that only one operation is used but rather that the number of operations is always constant.

2. Linear time, or O(n), indicates that the time it takes to run an algorithm grows in a linear fashion as n increases. In linear time, searching a list of 1,000 records should take roughly 10 times as long as searching a list of 100 records, which in turn should ~~be~~ take roughly 10 times as long as searching a list of 10 records.

Shot on OnePlus
2025.08.06 11:28

3. ~~Logarithmic~~ Quardratic Time: or $O(\log n^2)$, indicates that the time needed to run an algorithm grows as a ~~logarithm~~ square of n. For example - in a selection sort algorithm, a list is sorted by repeatedly finding the min^m value in the unsorted part of the list and placing this value in the begining. Because the no. of operations needed to find min^m value in the list grows as the length n of the list grows, and the no. of values that must be sorted also grows with n, the total no. of operations grows with n^2. These algorithms grows much faster than those grow in linear time.

4. Logarithmic time, or $O(\log n)$, indicates that the time needed to run an algorithm grows as a logarithm of n.

For. ex.- Binary search on a sorted list is performed, The list is searched by dividing it in half repeadedly until the desired element is found. The no. of divisions necessary to find the element grows with the logarithm of n in base 2 rather than proportionally to n. $n(\log n)$ is slower growth rate than $O(n)$; thus, these algorithms have lower time complexity than linear time algorithms.

Shot on OnePlus
2025.08.06 11:29

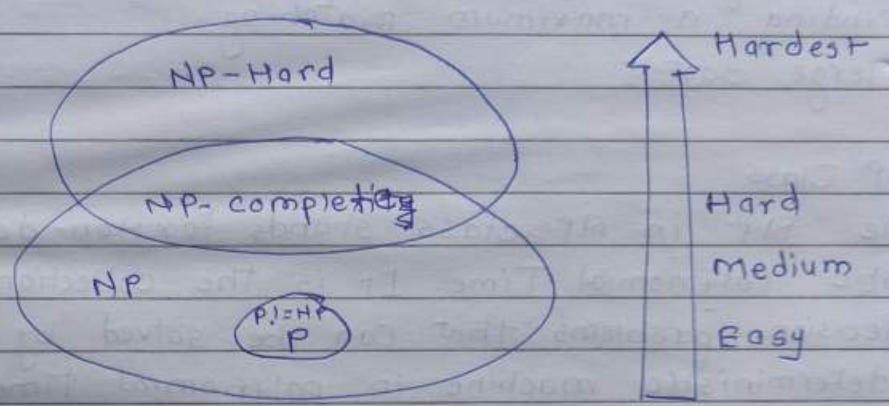
P, NP, NP-hard, NP-complete

Polynomial & Non-polynomial time

Polynomials only involve operations of addition, subtraction, multiplication and non-negative integer exponents.

Non-polynomials can involve other operations or types of exponents.

A polynomial-time algo. is one whose running time grows as a polynomial function of the size of its input.



Easy → P
Medium → NP
Hard → NP-complete
Hardest → NP-Hard

P Class

The P in the P class stands for Polynomial time. It is the collection of decision problem (problems with a "yes" or "no" answer) that can be solved by a deterministic machine in a polynomial time.

Shot on OnePlus
2025.08.06 11:29

Features:

- The solution to P-problems is easy to find.
- P is often a class of computational problems that are solvable and tractable.

Tractable means that the problems can be solved in theory as well as in practice. But the problems that can be solved in theory but not in practice are known as intractable.

This class contains many problems:

1. Calculating the greatest common diviser
2. Finding a maximum matching.
3. Merge sort.

NP Class

The NP in NP class stands for Non-deterministic Polynomial Time. It is the collection of decision problems that can be solved by a non-deterministic machine in polynomial time.

Features:

The solutions of the NP class are hard to find since they are being solved by a non-deterministic machine but the solutions are easy to verify.

Shot on OnePlus
✨ | 2025.08.06 11:29

Insertion - O(n) - Best
O(n²) - Worst
Selection - O(n²) - Best-worst-Avg

NP-hard Class

An NP-hard problem is at least as hard as the hardest problem in NP and it is a class of problems such that every problem in NP reduces to NP-hard.

Features:
All NP-hard problems are not in NP
It takes a long time to check them.

Shot on OnePlus
✨ | 2025.08.06 11:29

3. Master Method : (only for specific problem, faster method than substitution method).

Master Method is a direct way to get the solution.

This Method works only for the following types of Recurrences or for Recurrences that can be transformed into the following type ;

$$T(n) = a\,T(n/b) + F(n)$$

where, $a \geq 1$ & $b > 1$

e.g. Recurrence Relations.

1. $T(n) = T(n) + n^2$ ✗
2. $T(n) = 8T(n/2) + n^2$ ✓
3. $T(n) = T(n/2) + c$ ✓

Among this only 2nd & 3rd can be solve using master theorem. [$\because b > 1$ & in 1st, $b = 1$]

Solution : $T(n) = n^{\log_b a}\,[u(n)]$

$u(n)$ depends on $h(n)$

$$\boxed{h(n) = \frac{F(n)}{n^{\log_b a}}}$$

Shot on OnePlus
2025.08.06 11:34